

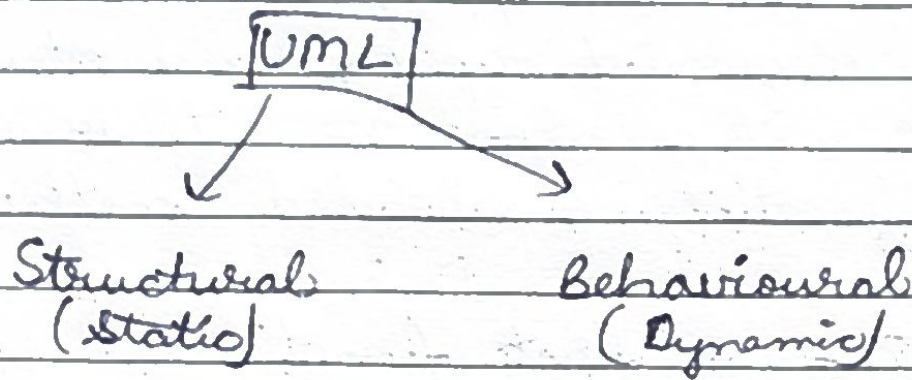
* UML Diagrams :-

Unified Modeling Language (UML) Diagrams are a visual way to represent the design of a software application. Instead of long, monotonous paragraphs of documentation, diagrams allow developers to intuitively explain:

- Components → The objects or entities within the system.
- Interaction → How those components connect and send messages to each other.

Date.....

* Types of UML Diagrams :-



Structural → Define the physical structure of your applications - what components exist and how they are connected.

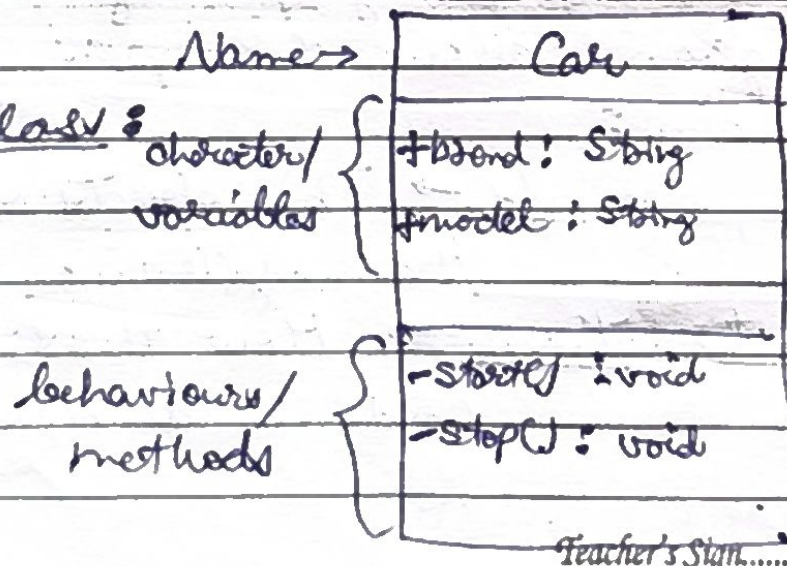
Behavioural → Define how components interact, send messages, and call methods during runtime.

★ In total, there are 14 types, but you only need to master only 2 as they are most frequently used.

* UML Class Diagrams :-

A class diagram is a static structure diagram that shows system's classes, their attributes, methods, and relationship between them.

→ Representation of a class :-



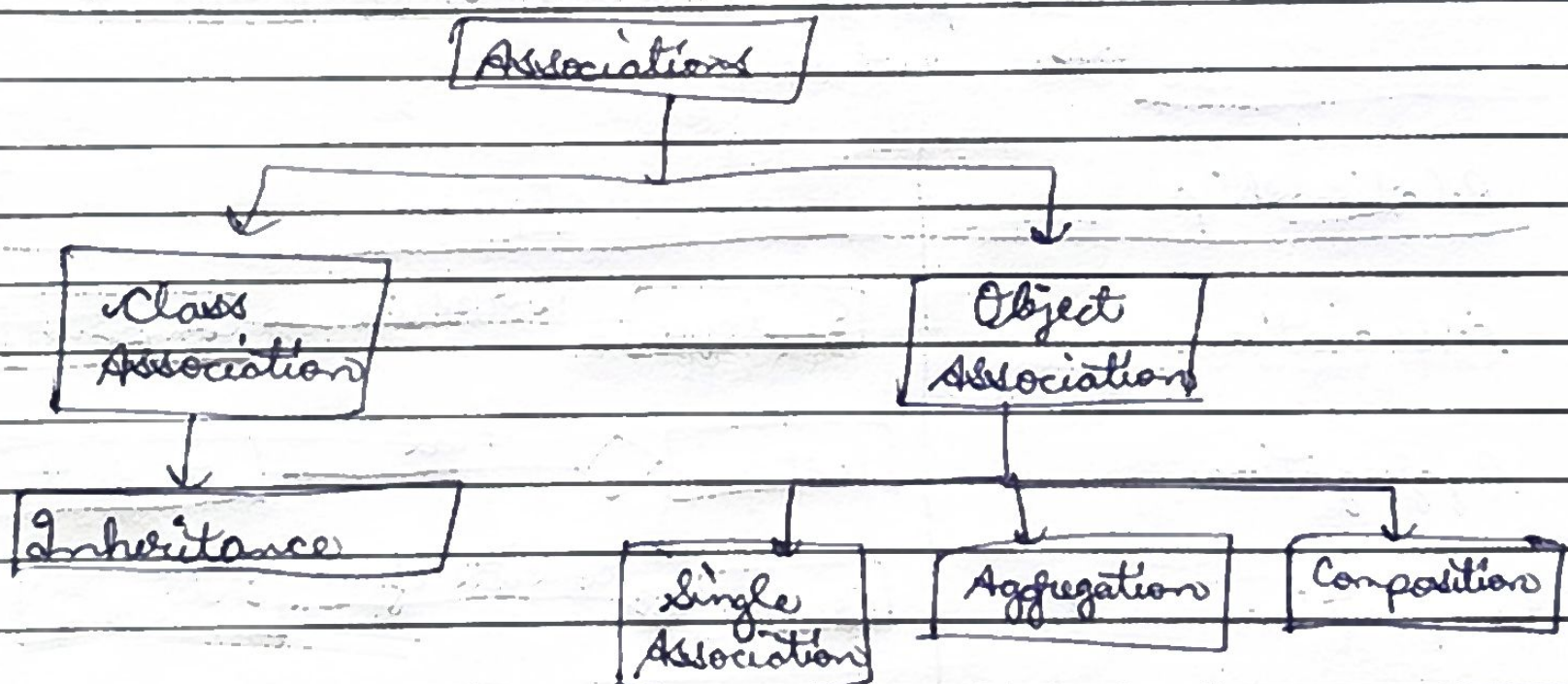
Date.....

Visibility markers that prefix each member:

- + → public (accessible everywhere)
- → private (accessible in class & child classes)
- ## → protected (accessible only in class)
- ~ → package (folder)

→ Class Association:

It describes how diff. classes interact or depend on one another.



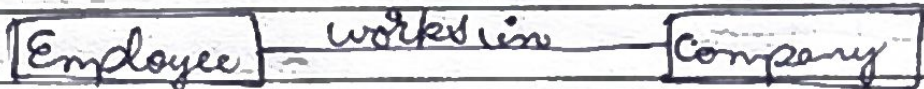
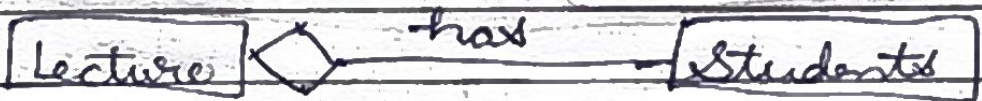
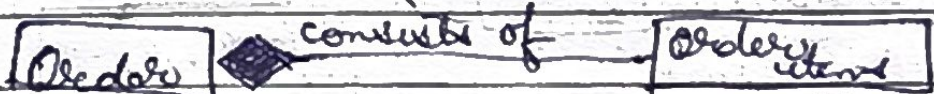
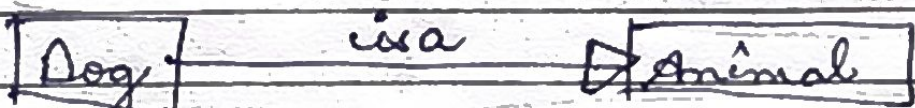
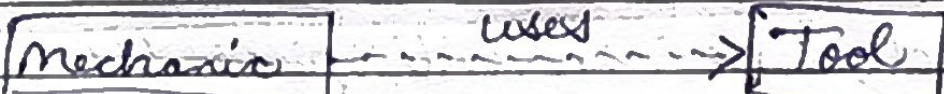
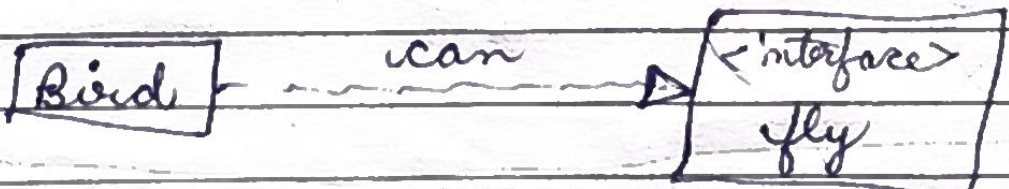
Inheritance (IS-A Relationship) →

It allows a child class to inherit properties and methods from a parent class.

Date.....

Object Association (HAS-A Relationship) →

- Simple Association (Weakest) : A basic connection between two independent entities.
- Aggregation (Moderate) : A "contains" relationship where one object contains others, but the parts can exist independently of the container.
- Composition (Strongest) : A strict relationship where the parts cannot exist without the whole.

<u>Relationship</u>	<u>Notation</u>
<u>Association</u>	
<u>Aggregation</u>	
<u>Composition</u>	
<u>Inheritance</u>	
<u>Dependency</u>	
<u>Implementation</u>	

Date.....

Sagar 02/4

* UML Sequence Diagrams :-

It is a behavioural diagram that shows how objects interact over time to accomplish a task. It captures the dynamic, time-ordered flow of messages between participants.

Key ideas: time flows top $\rightarrow$ bottom and each participant gets a vertical lifeline.

* Core Elements $\rightarrow$

class/object

Object



Actor

(User/
Another system)

Lifeline

messages

Activation
bar

Destruction

Date.....

Object → A class instance participating in the interaction.

Lifeline → Represents an object's existence over time.

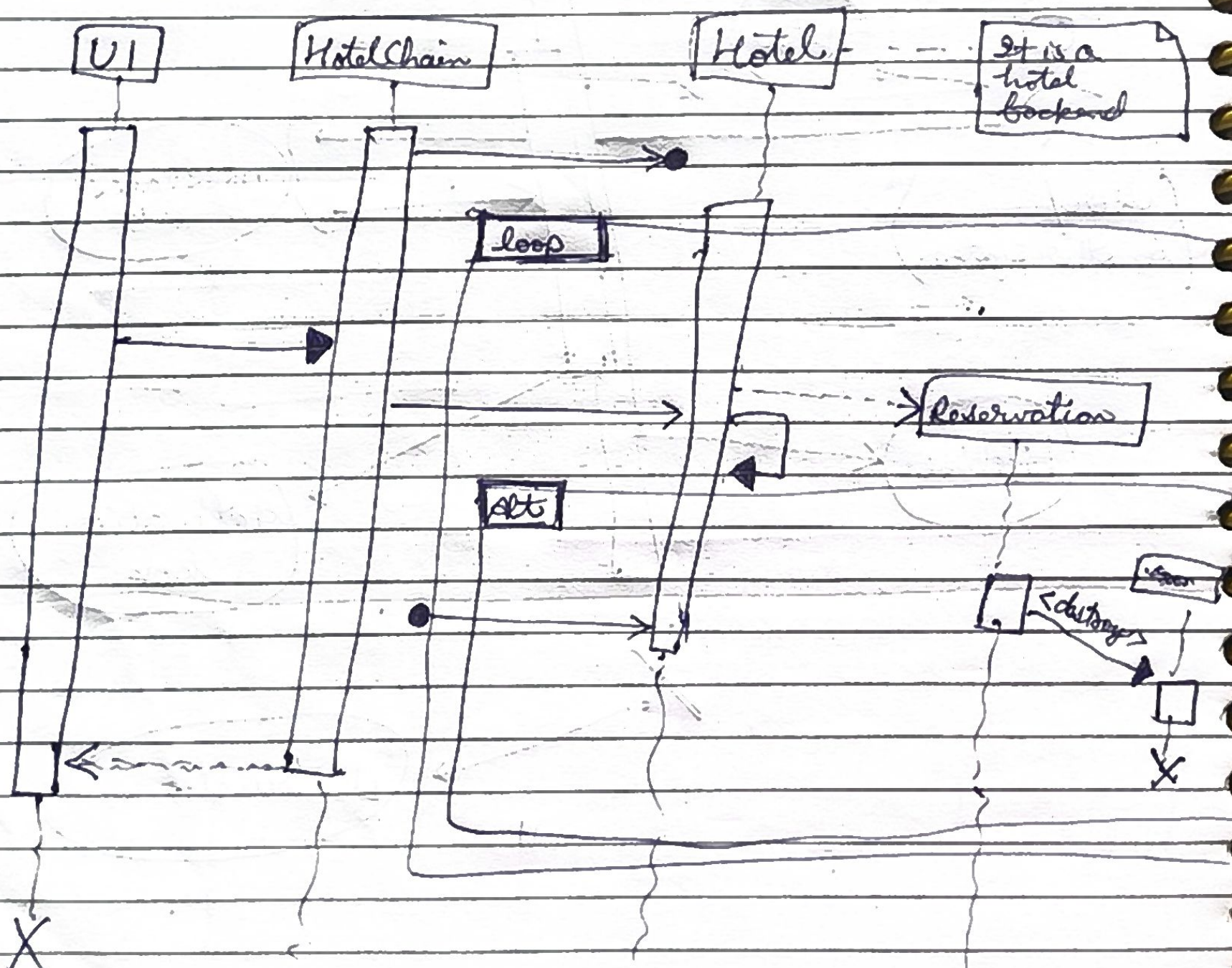
Activation box → Period during which object is actively executing.

Message → A method call or signal sent between objects.

Actor → External entity (user, another system).

Destruction (X) → object is destroyed at that point.

• Message Types →



Date.....

Sagarika

Synchronous call → Sender waits for a reply



Asynchronous call → Sender does not wait for response



Reply → Response sent back to caller



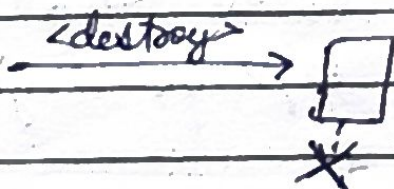
Self-Message → Object calls its own method



Create → Instantiates a new object



Destroy → Terminates / deletes an object



Lost → It happens when message is sent, but there is no one to receive it. It never reaches its destination.



Date.....

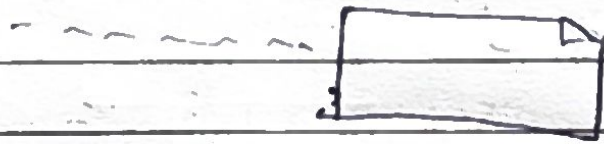
Found → It happens when message is received but don't know who sent it.



Duration Message → When a message reaches after some time or there is a time diff between sending & receiving.



Note → It is like a comment that is used to attach remarks / information to elements.



• Combined Fragments (Control Flow) →

These are rectangular boxes that wrap a group of messages to express logic:

Fragment	Keyword	Meaning
Alternative	alt	if/else branching — only executes with true branch
Option	opt	optional — executes only if condition is true
APCO		Teacher's Sign.....

Date.....

Break	break	Exceptional exit - if condition true, break
negative	neg	Invalid sequence - tells a flow that must never happen. (Documentation)
Loop	loop	while/for - repeats as long as cond ⁿ is true
Parallel	par	Concurrent execution - all operands execute same time
Critical Region	critical	only one thread can execute at a time, everyone waits.
Strict Sequencing	strict	operands must execute in a strict defined order
Weak Sequencing	seq	should follow order, but interleaving is allowed.
Ignore	ignore	Suppress Messages
Consider	consider	opposite of ignore
Assertion	asserts	Mandatory invariant - asserts that shown sequence must always occur, others are invalid.

Date.....

Reference	ref	External reference - a separate interaction defined in another diagram.
Sequence Diagram	sd	Nested diagram - another diagram inside current diagram.

★★

Class Diagrams : Use these when you need to define the 'static blueprint' of your system. They represent internal structure of classes, their attributes, methods, and relationships (like inheritance & composition).

Sequence Diagrams : Use these when you need to visualize the 'dynamic flow of control'. They are perfect for documenting complex business logic, multi-object interactions, or debugging how a request travels through different system layers (like User → ATM → Transaction → Account).